The Physics Behind a Computational Simulation of Diffusion

Mario Borilov

May 2026

1 Introduction

This report aims to explain the physics and mathematics describing the diffusion process and the technique used to simulate this physical process.

2 Useful mathematics

Before covering the equations that are used for describing the diffusion we should cover some of the mathematics necessary for these equations. The first mathematical tool which is necessary for the diffusion equation is the Nabla operator.The Nabla operator is operator that takes a function and returns a vector with components the spatial derivatives of the function to its arguments.This vector is called gradient of the function and its notation is:

$$grad(f) = \nabla(f) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{bmatrix}$$

So only the Nabla operator notation is: $\nabla = \begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{bmatrix}$

The dot product of two vectors with x, y, z components can be defined as:

$$\vec{a} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \ \vec{b} = \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix} \ \vec{a}.\vec{b} = a_x b_x + a_y b_y + a_z b_z$$

The Nabla operator has the properties of a vector so the vector operations can be applied to the Nabla operator.Cross and dot product can be defined for the Nabla operator.When we take the scalar square of a Nabla operator we get:

$$\nabla^2 = \nabla.\nabla = \frac{\partial}{\partial x}\frac{\partial}{\partial x} + \frac{\partial}{\partial y}\frac{\partial}{\partial y} + \frac{\partial}{\partial z}\frac{\partial}{\partial z} = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}$$

The scalar square of the Nabla operator is called Laplacian and it is also a operator that takes a function and gives back the sum of the second partial derivatives of the function: $\nabla^2(f) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}$ When $f = f(x,y) => \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

3 Diffusion Equation

The diffusion equation is partial differential equation that describes the concentration C of given substance(number of particles in one cubic meter) as function of time and the spacial coordinates: $\frac{\partial C}{\partial t} = k\nabla^2 C$

Where $k = k(T)$ is called diffusion constant that shows how high is the rate of diffusion of the given substance in the material medium.Because the temperature indicates the intensity of the chaotic thermal movement of the particles the diffusion constant depends on temperature. The equation giving the closest solution for k compared to the experimental results is called the Arrhenius relation: $k = k_0.e^{\frac{-E_A}{RT}}$ Were k_0 is the maximal value of k, E_A is so called diffusion activation energy, R is the universal gas constant and T is the absolute temperature.

4 Computational Simulation

For simplicity we simulate the process only in 2D (x, y plane)-the concentration $C = C(x,y)$ and also that the diffusion coefficient is constant value throughout the process. In the beginning of our program we import all libraries that are needed for the simulation. We import NumpY and pyplot and FuncAnimation from matplotlib. The we set the length of the x and y axes. In order to solve the partial differential equation numerically we divide x and y in small discrete pieces using large number of points to divide the y ad x axes into these small pieces. the length of this discrete pieces can be found by dividing the length of the axes by the number of all but one points, because the number of the pieces is with one lower than the number of the points. Then the mesh is created using the x and y coordinates in order to be possible to define the scalar field of the concentration. The initial scalar field of concentration is defined using exponential gaussian function and the coordinates of the highest initial concentration are set to be just in the center of the coordinate system. Because the concentration is scalar field (depends on the two coordinates) in the code is described by two dimensional array so each component of the array is the concentration in given point with certain coordinates. For the visualization of the scalar field is used pcolormesh. The simulation shows the concentration not only as function of the spacial coordinates but also of time. In order to do that we use "update" function that updates the values of the concentration in every

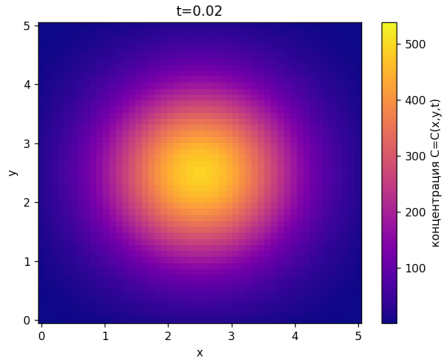

Figure 1: The concentration at 0.02s after the beginning of the simulation

point in space with the change of the frames. The values of the scalar field are updated using the equation: $C(x, y, t + \partial t) = C(x, y, t) + \partial C$ $\partial C = k\partial t\nabla^2(C) = k\partial t(\frac{\partial^2 C}{\partial x^2} + \frac{\partial^2 C}{\partial y^2})$ The second partial derivatives can be approximated using very small finite discrete pieces δx, δy via Taylor's expansion:

$\frac{\partial^2 C}{\partial x^2} \approx \frac{C(x,y)-2C(x+\delta x,y)+C(x+2\delta x,y)}{(\delta x)^2}$

$\frac{\partial^2 C}{\partial y^2} \approx \frac{C(x,y)-2C(x,y+\delta y)+C(x,y+2\delta y)}{(\delta y)^2}$

For the updated value after time interval δt passes we get:

$C(x, y, t+\partial t) = C(x, y, t)+k\delta t(\frac{C(x,y)-2C(x+\delta x,y)+C(x+2\delta x,y)}{(\delta x)^2}+$

$+\frac{C(x,y)-2C(x,y+\delta y)+C(x,y+2\delta y)}{(\delta y)^2})$

The updates of the scalar field are animated using FuncAnimation from matplotlib.animation using the output of the "update" function.